QA Automation Practical Task

Yahoo Signup Form Testing using Playwright

This task evaluates practical web automation skills using Playwright. Candidates should demonstrate reliable locator strategy, validation coverage, reusable test structure, test data management, debugging artifacts, and clear reporting.

1. Objective

Automate tests for the Yahoo account creation flow at **https://login.yahoo.com/account/create**. The goal is to validate important user journeys, field-level validation, error handling, and test framework quality using Playwright.

Because this is a live third-party website, the task should avoid creating real accounts or bypassing anti-bot, CAPTCHA, OTP, phone verification, or email verification controls. Tests must stop safely before irreversible account creation or external verification steps.

2. Required Technology

- **Playwright** with JavaScript/TypeScript or Python. TypeScript is preferred, but Python is acceptable.
- Chrome/Chromium must be covered. Firefox and WebKit coverage should be included where feasible.
- Use Playwright Test runner features such as fixtures, projects, traces, screenshots, videos, retries, and HTML reports.
- Use a clean project structure with package scripts or equivalent commands to install and run the tests.

3. Form Areas to Test

- First name and last name
- Email address or desired Yahoo username
- Password
- Country code and mobile phone number
- Birth month, day, and year, if present in the current UI
- Continue / submit action
- Inline validation messages, disabled states, focus behavior, and error summaries where available

4. Core Test Requirements

Area	Expected Coverage
Required fields	Verify required-field validation for all mandatory inputs. Include blank form submission and field-by-field validation.
Email / username	Check invalid formats, unavailable or malformed usernames if supported by the UI, boundary lengths, and whitespace handling.
Password	Validate minimum length, weak password patterns, missing uppercase, missing lowercase, missing number or special character if enforced, and show/hide password behavior.
Phone number	Validate country-specific formatting, invalid characters, too short, too long, and country code changes.
Date of birth	Validate missing date values, unrealistic ages, underage restrictions if displayed, invalid combinations, and boundary dates.

Submission flow	Use valid-looking test data and verify that the form proceeds to the next expected safe step without completing external verification or creating an account.

5. Practical Complexity to Add

Candidates should not submit a single linear script. The solution should behave like a small production-quality automation suite.

- **Page Object Model:** Create a page object or screen object for the signup page. Keep selectors and user actions separate from assertions.
- **Data-driven testing:** Store positive and negative datasets in JSON, CSV, fixtures, or parameterized test cases.
- **Reusable fixtures:** Use Playwright fixtures for browser context setup, test data generation, and common navigation.
- **Cross-browser execution:** Configure at least Chromium and one additional browser project. Explain any browser-specific limitation.
- **Mobile viewport:** Add at least one test project for a mobile-sized viewport and verify the form remains usable.
- **Network awareness:** Capture failed requests or relevant response status codes during validation and include them in logs when a test fails.
- **Artifacts:** Enable screenshots, trace viewer output, videos on failure, and HTML report generation.
- **Accessibility smoke check:** Verify labels, keyboard navigation, focus order, and visible error messages for at least the main fields.
- **Stability:** Avoid fixed sleeps. Use Playwright auto-waiting, assertions, and explicit waits for meaningful UI states.
- **Safe execution:** Do not automate OTP, CAPTCHA, phone verification, email verification, or final account creation.

6. Required Test Scenarios

1. Verify that submitting an empty form displays validation messages for all required fields.

2. Verify that each required field displays a validation message when cleared after user interaction.

3. Verify invalid email / username formats, including missing domain, invalid characters, leading/trailing spaces, and boundary lengths.

4. Verify invalid password formats, including too short, no uppercase, no lowercase, no numeric or special character if required, common weak values, and whitespace-only values.

5. Verify mobile number validation for invalid characters, too-short numbers, too-long numbers, and country code changes.

6. Verify date of birth validation for missing values, invalid age, boundary age, and invalid combinations if supported by the UI.

7. Verify that validation messages disappear or update correctly after fields are corrected.

8. Verify keyboard-only interaction: tab order, focus visibility, field completion, and triggering validation without using the mouse.

9. Verify responsive behavior on a mobile viewport and ensure fields and validation messages are visible without overlap.

10. Verify the safe positive path using generated test data and stop at the next expected verification step without completing real account creation.

11. Verify that test failure artifacts are generated by intentionally documenting how screenshots, traces, and reports are captured.

12. Verify that selectors are robust by preferring role, label, placeholder, accessible name, or test-friendly locators where available instead of brittle XPath/CSS chains.

7. Suggested Project Structure

The exact structure may vary, but the submission should be easy to install, understand, and run.

```
qa-playwright-task/
  package.json
  playwright.config.ts
  tests/
    signup.validation.spec.ts
    signup.positive.spec.ts
    signup.mobile.spec.ts
    signup.accessibility.spec.ts
  pages/
    SignupPage.ts
  fixtures/
    testData.ts
  data/
    signup-negative-data.json
  reports/
  README.md
```

8. Example Commands

```
npm install
npx playwright install
npx playwright test
npx playwright test --project=chromium
npx playwright show-report
npx playwright show-trace path/to/trace.zip
```

9. Reporting Requirements

- Summarize the test approach and framework structure.
- List executed scenarios and their status.
- Document defects, unexpected behavior, flaky areas, or blocked scenarios.
- Include screenshots, traces, videos, console logs, or network observations for failures.
- Explain any limitation caused by CAPTCHA, OTP, anti-automation behavior, or live-site changes.
- Mention how to run tests locally and in CI.

10. Deliverables

- Complete Playwright test project source code.
- README with setup instructions, run commands, assumptions, and limitations.
- Brief test report in Markdown, PDF, or HTML.
- Sample Playwright HTML report, screenshots, and trace files for at least one passing and one failing or intentionally demonstrated validation scenario.
- Optional: CI configuration, such as GitHub Actions, that runs the suite headlessly and uploads Playwright artifacts.

11. Evaluation Criteria

Criterion	Weight
Functional coverage of validation and key flows	25%
Playwright usage, assertions, fixtures, and waits	20%
Framework structure, maintainability, and Page Object Model	20%
Stability, artifacts, and debugging quality	15%
Data-driven approach and realistic edge cases	10%
Report quality, clarity, and safe handling of live-site limitations	10%

12. Important Notes

- The Yahoo UI may change. Tests should be written to be resilient and easy to update.
- Do not use fixed waits such as sleep(5000) unless justified for debugging and removed from final tests.
- Do not use real personal phone numbers, real user credentials, or production email addresses.
- Do not attempt to bypass security, rate limits, CAPTCHA, OTP, or verification systems.
- A strong submission explains what was automated, what could not safely be automated, and why.

End of task.